Digital Signal Processing Exercise Sheet 3

LTI Systems

Script: Chapter 3 (Linear Time-Invariant Systems) **with solutions**

Learning goals

- Compute convolutions by hand (graphically and in tables) and in C; verify the algebraic properties of convolution numerically.
- Derive impulse and step responses of the two workhorse smoothers – moving average (FIR) and exponential smoother (IIR) – from their difference equations.
- Quantify the noise-versus-lag trade-off of a smoother (noise gain, mean delay, ramp lag) and relate it to the step response.
- Apply LTI filters to the per-tooth engine speed in real time and understand why the missing-tooth gap must be handled *before* the filter (time invariance in the tooth domain).

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. If you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet03/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o conv conv.c -lm
./conv
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

A discrete-time LTI system is completely described by its impulse response $h[n] = S\{\delta[n]\}$; the output for any input is the convolution

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Two smoothers are used throughout the course (written for the per-tooth speed $r[k]$, but valid for any sequence):

$$\text{MA (FIR):}\quad y[n] = \frac{1}{N}\sum_{k=0}^{N-1} x[n-k] = y[n-1] + \frac{x[n] - x[n-N]}{N},$$

$$\text{EMA (IIR):}\quad y[n] = a\,y[n-1] + (1-a)\,x[n] = y[n-1] + \alpha\,(x[n] - y[n-1]), \qquad \alpha = 1-a,$$

called moving average and exponential smoother (exponential moving average). Useful characteristics of a smoother with $\sum_n h[n] = 1$: the *noise gain* $\sum_n h^2[n]$ (variance reduction for white noise) and the *mean delay* $D = \sum_n n\,h[n]$.

Exercise 3.1 Convolution and smoothers by hand

PAPER

a) Compute $y = x * h$ for $x[n] = \{\underline{1}, 2, 0, -1\}$ and $h[n] = \{\underline{1}, -1, 0.5\}$ (underlined: $n = 0$) twice: graphically (flip and shift h) and with a table of products $x[k]h[n-k]$. Check the result with $\sum_n y[n] = \sum_n x[n] \cdot \sum_n h[n]$ and explain why this identity holds.

b) Determine impulse response $h[n]$ and step response $s[n]$ of the MA of length N and of the EMA (from the difference equation by iteration and from $s[n] = \sum_{k\le n} h[k]$). After how many samples does each reach 50 % and 90 % of a step for $N = 16$ and $a = 15/17$?

c) Compute noise gain and mean delay D of both smoothers. Show that for a ramp input $x[n] = c\,n$ every smoother with $\sum h = 1$ has the steady-state output $y[n] = c\,(n - D)$, and that $D = \sum_{n\ge 0}(1 - s[n])$, i.e. the ramp lag can be read off the step response. Which a gives the EMA the same noise gain as the MA of length N? Compare the mean delays.

d) Show that $x[n] = \mathrm{e}^{\mathrm{j}\Omega n}$ is an eigenfunction of every LTI system, $y[n] = H(\Omega)\,x[n]$ with $H(\Omega) = \sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. Evaluate $|H|$ for MA_{16}, MA_{30} and the EMA with $a = 15/17$ at $\Omega = 2\pi/30$. (On the 60-tooth wheel, sampled once per 6°, this is the 2nd engine order – the speed ripple caused by the combustions.)

Solution

a) Table ($y[n] = \sum_k x[k]h[n-k]$; rows k, columns n):

$x[k]\,h[n-k]$	$n=0$	1	2	3	4	5
$k=0,\ x=1$	1	−1	0.5			
$k=1,\ x=2$		2	−2	1		
$k=2,\ x=0$			0	0	0	
$k=3,\ x=-1$				−1	1	−0.5
$y[n]$	1	1	−1.5	0	1	−0.5

Graphically: $h[n-k]$ is h flipped and moved to position n; $y[n]$ is the sum of the products where flipped h and x overlap. Length $4+3-1 = 6$. Check: $\sum y = 1 = (1+2+0-1)(1-1+0.5) = 2\cdot 0.5$. Reason: $\sum_n \sum_k x[k]h[n-k] = \sum_k x[k] \sum_m h[m]$ (substitute $m = n - k$). The program of Exercise 3.2 prints `y = 1 1 -1.5 0 1 -0.5`.

b) MA: $h[n] = 1/N$ for $0 \le n \le N-1$, else 0; $s[n] = (n+1)/N$ for $0 \le n < N$, $s[n] = 1$ for $n \ge N-1$ (linear ramp, finite). EMA: iterating $y[n] = a\,y[n-1] + (1-a)\delta[n]$ gives $h[n] = (1-a)a^n u[n]$; $s[n] = (1-a)\sum_{k=0}^{n} a^k = 1 - a^{n+1}$ (exponential, never exactly 1). For $N = 16$: MA reaches 50 % at $n = 7$ (8/16), 90 % at $n = 14$ (15/16; $14/16 < 0.9$). EMA, $a = 15/17$: $a^{n+1} \le 0.5 \Leftrightarrow n+1 \ge \ln 0.5/\ln a = 5.55$, i.e. $n = 5$; 90 %: $n+1 \ge \ln 0.1/\ln a = 18.4$, $n = 18$. The EMA reacts faster initially but needs longer to settle.

c) MA: $\sum h^2 = N \cdot \frac{1}{N^2} = \frac{1}{N}$, $D = \frac{1}{N}\sum_{n=0}^{N-1} n = \frac{N-1}{2}$. EMA: $\sum h^2 = (1-a)^2 \sum a^{2n} = \frac{(1-a)^2}{1-a^2} = \frac{1-a}{1+a}$, $D = (1-a)\sum n a^n = \frac{a}{1-a}$. Ramp: $y[n] = \sum_k h[k]\,c(n-k) = c\,n\sum_k h[k] - c\sum_k k h[k] = c(n-D)$ (for n beyond the length of h; for the EMA asymptotically). With $1 - s[n] = \sum_{k>n} h[k]$: $\sum_{n\ge 0}(1-s[n]) = \sum_k k\,h[k] = D$. Equal noise: $\frac{1-a}{1+a} = \frac{1}{N} \Rightarrow a = \frac{N-1}{N+1}$, $\alpha = \frac{2}{N+1}$ (the usual "N-period EMA"). Then $D_{\mathrm{EMA}} = \frac{(N-1)/(N+1)}{2/(N+1)} = \frac{N-1}{2} = D_{\mathrm{MA}}$: *for equal white-noise reduction both smoothers have exactly the same ramp lag.* They differ in the shape of the step response (b) and in how they suppress periodic disturbances (d). For $N = 16$: $a = 15/17$, $D = 7.5$ samples.

d) $y[n] = \sum_k h[k]\mathrm{e}^{\mathrm{j}\Omega(n-k)} = \mathrm{e}^{\mathrm{j}\Omega n}\sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. MA: geometric sum, $|H| = \left|\frac{\sin(N\Omega/2)}{N\sin(\Omega/2)}\right|$; EMA: $H = \frac{1-a}{1-a\mathrm{e}^{-\mathrm{j}\Omega}}$. At $\Omega = 2\pi/30$: MA_{16}: $\frac{\sin(16\pi/30)}{16\sin(\pi/30)} = \frac{0.9945}{1.6724} = 0.595$; MA_{30}: $\sin(\pi) = 0 \Rightarrow |H| = 0$

– the MA over one full period removes the ripple *completely* (the sum over a period of a zero-mean periodic signal vanishes); EMA: $\frac{0.1176}{|1-0.8824(0.9781-0.2079\mathrm{j})|} = \frac{0.1176}{0.2289} = 0.514$. These values are confirmed by the simulation in Exercise 3.3 a).

Exercise 3.2 Convolution engine

JSLINUX

Program: `code/jslinux/sheet03/conv.c`.

a) Implement `conv(x, nx, h, nh, y)` (full linear convolution, output length $n_x + n_h - 1$, no out-of-range accesses) and check it with the example of Exercise 3.1 a).

b) Verify commutativity, associativity and distributivity with random sequences of lengths 200, 50 and 30. Why are the deviations not exactly zero? Why might they be exactly zero for "nice" test numbers?

c) Implement the exponential smoother recursively (`exp_smooth()`) and compare it with the convolution by its impulse response truncated to L samples ($a = 0.9$, $L = 10 \ldots 160$). Relate the error to a^L and compare the number of multiplications per output sample. Verify the step response formula of Exercise 3.1 b).

d) Convolve the impulse response of an 8-point MA with itself. Explain the shape and the mean delay of the result and compare the step responses of one MA and of the cascade.

Solution

a) Key part of `code/solutions/jslinux/sheet03/conv.c` – the summation limits make sure that both $x[k]$ and $h[n-k]$ exist:

```
static int conv(const double *x, int nx, const double *h, int nh, double *y)
{
    int n, k, ny = nx + nh - 1;
    for (n = 0; n < ny; n++) {
        int k0 = n - nh + 1 > 0 ? n - nh + 1 : 0;    /* h index n-k < nh  */
        int k1 = n < nx - 1 ? n : nx - 1;            /* x index k < nx    */
        double s = 0.0;
        for (k = k0; k <= k1; k++) s += x[k] * h[n - k];
        y[n] = s;
    }
    return ny;
}
```

Output: `y = 1 1 -1.5 0 1 -0.5` (as by hand).
b), c) Output:

```
(b) commutativity   max|x*h - h*x|             = 2.66e-15
    associativity   max|(x*h)*g - x*(h*g)|     = 1.07e-14
    distributivity  max|x*(h+g) - (x*h + x*g)| = 5.33e-15

(c) exponential smoother a = 0.90, 1000 samples
     L   max|recursive - conv|   a^L      mult./sample
    10   3.774e-01           3.487e-01   10 (recursion: 2)
    20   1.316e-01           1.216e-01   20 (recursion: 2)
    40   1.600e-02           1.478e-02   40 (recursion: 2)
    80   2.365e-04           2.185e-04   80 (recursion: 2)
   160   5.166e-08           4.773e-08   160 (recursion: 2)
    step response vs. 1 - a^(n+1): max deviation 1.11e-16
```

The properties hold exactly in real arithmetic; in floating point the sums are evaluated in a different order and rounded differently (deviations of a few 10^{-15}, i.e. some units of the last place relative to values of order 10). With test numbers that are short binary fractions (e.g. multiples of 2^{-15}) all products and sums are exact and the deviation is exactly 0 – a test that proves nothing about rounding. (c): The truncated impulse response misses the tail $\sum_{n \geq L} h[n] = a^L$;

for an input of magnitude ≈ 1.1 (step + noise) the error is $\approx 1.1\,a^L$, as observed. The recursion needs 2 multiplications per sample for an *infinite* impulse response, the convolution L – the reason why IIR filters are cheap. The recursive step response matches $1 - a^{n+1}$ to rounding precision.
d) $h_{\mathrm{MA8}} * h_{\mathrm{MA8}} = \frac{1}{64}\{1, 2, \ldots, 7, 8, 7, \ldots, 2, 1\}$ (length 15): the overlap of two rectangles of width 8 grows linearly and shrinks again – a triangle. Sum 1, mean delay $7 = 2 \cdot 3.5$ (mean delays add under convolution, like the means of independent random variables). The step response of the cascade is a smooth S-curve (piecewise parabolic) reaching 1 after 15 samples instead of the linear ramp of one MA reaching 1 after 8 samples: better high-frequency suppression, twice the delay.

Exercise 3.3 Smoothing the RPM estimate

WOKWI

The per-tooth speed $r[k] = 1/T_k$ of Sheet 1 contains the real speed fluctuation (2nd order, ±1 %) and timer quantisation. We smooth it with an MA and an EMA in the *tooth domain* (one sample per tooth). Three ways to treat the gap (period $\approx 3T$) are compared: `GAP_MODE` 0 – use $1/T_{\mathrm{gap}}$ like a normal value; 1 – correct it to $3/T_{\mathrm{gap}}$ (one sample); 2 – insert $3/T_{\mathrm{gap}}$ three times (one sample per 6°).

a) *Offline prediction* (JSLINUX, `code/jslinux/sheet03/rpm_offline.c`). The program simulates the crank edges with 0.5 µs time stamps. Implement the gap handling, `moving_average()` (running sum) and `exp_smoother()`. Report: at constant 3000 rpm mean, standard deviation and min/max of raw, MA_{16}, MA_{30}, $\mathrm{EMA}_{2/17}$ for all three gap modes; for a step 3000→5000 rpm the 50 %/90 % times in samples for $N = 8, 16, 32$ and $\alpha = 2/(N+1)$; for a ramp of 3000 rpm/s the lag error in rpm, in samples and in ms. Compare with Exercise 3.1.

b) *Real time* (WOKWI, `code/wokwi/sheet03_rpmfilter/`). The ISR pushes tooth periods into a FIFO; `loop()` computes and filters the speed. Implement the same three steps in the sketch. It prints one line per revolution and, whenever the raw speed jumps by more than 150 rpm between two teeth, a capture block of 128 samples (32 before the jump). Start at 3000 rpm and *click* the speed slider at 5000 rpm. Determine the 50 %/90 % points of both filters from the capture and the ramp lag $D = \sum(1 - s[n])$. (Dragging the slider produces a staircase of steps, not a ramp – why is the step response sufficient anyway?)

c) Repeat with `GAP_MODE` 0 and 1. Explain the per-revolution values and the dips in the capture. Why does the gap violate the time invariance of the tooth-domain processing, and why is only mode 2 compatible with the exact ripple cancellation of MA_{30}?

d) Design: the injection timing needs the speed during an acceleration of 5000 rpm/s at 1000 rpm with an error below 0.5 %, while the 2nd-order ripple should be suppressed. Is this possible with an MA in the tooth domain? What do real ECUs do (keyword: *segment time*)?

Solution

a) Key parts of `code/solutions/jslinux/sheet03/rpm_offline.c`:

```
                /* ---- (a) per-tooth speed, gap handling ---------------- */
                v = 1.0 / (T * TICK);                       /* rpm = 1/T[s] */
                if (gap && gap_mode != GAP_NAIVE) v *= 3.0;
                if (gap && gap_mode == GAP_THREE) rep = 3;
```

```
static void moving_average(const double *x, int n, int N, double *y)
{
    double s = N * x[0];
    int k;
    for (k = 0; k < n; k++) {
        s += x[k] - (k >= N ? x[k - N] : x[0]);
        y[k] = s / N;
```

```
    }
}

/* exponential smoother y[k] = y[k-1] + alpha (x[k] - y[k-1]),
   pre-loaded with x[0]                                          */
static void exp_smoother(const double *x, int n, double alpha, double *y)
{
    double s = x[0];
    int k;
    for (k = 0; k < n; k++) {
        s += alpha * (x[k] - s);
        y[k] = s;
    }
}
```

Results (constant 3000 rpm, after settling):

```
gap handling: naive gap (2842 samples)
 raw        mean  2966.4  std 260.19  min   996.5  max  3030.3
 MA16       mean  2966.4  std  58.09  min  2859.7  max  3017.8
 MA30       mean  2966.3  std  32.88  min  2932.2  max  3000.0
 EMA 2/17   mean  2966.4  std  57.58  min  2750.7  max  3014.3
gap handling: gap once (2842 samples)
 raw        mean  3000.3  std  21.53  min  2967.4  max  3030.3
 MA16       mean  3000.3  std  12.19  min  2982.0  max  3017.8
 MA30       mean  3000.4  std   1.06  min  2998.7  max  3002.7
 EMA 2/17   mean  3000.3  std  10.73  min  2984.8  max  3015.5
gap handling: gap x3 (2940 samples)
 raw        mean  2999.9  std  21.27  min  2967.4  max  3030.3
 MA16       mean  3000.0  std  12.55  min  2982.0  max  3017.8
 MA30       mean  3000.0  std   0.04  min  2999.9  max  3000.1
 EMA 2/17   mean  3000.0  std  10.85  min  2984.8  max  3015.5
```

With correct gap handling the raw std (21.3 rpm) is the ripple ($30/\sqrt{2}$). The attenuation factors $12.55/21.27 = 0.59$ (MA_{16}) and $10.85/21.27 = 0.51$ (EMA) agree with $|H| = 0.595$ and 0.514 of Exercise 3.1 d); MA_{30} in mode 2 leaves 0.04 rpm (timer quantisation only). Step and ramp (mode 2):

```
 MA8        50% after   4 samples, 90% after   7 samples
 EMA 2/9    50% after   2 samples, 90% after  10 samples
 MA16       50% after   8 samples, 90% after  14 samples
 EMA 2/17   50% after   5 samples, 90% after  18 samples
 MA32       50% after  16 samples, 90% after  28 samples
 EMA 2/33   50% after  11 samples, 90% after  37 samples

 MA16       lag error    6.8 rpm =  7.93 samples =  2.25 ms at 3523 rpm
 EMA 2/17   lag error    6.9 rpm =  7.98 samples =  2.27 ms at 3523 rpm
 MA32       lag error   13.8 rpm = 16.07 samples =  4.56 ms at 3523 rpm
 EMA 2/33   lag error   13.8 rpm = 16.04 samples =  4.55 ms at 3523 rpm
```

The step points match 3.1 b) (MA_{16} is exactly at 50 % at $n = 7$; the ripple decides whether 7 or 8 is reported). The ramp lag is $D + 0.5$: $(N-1)/2$ from the filter plus half a sample because $1/T_k$ is the mean speed over the tooth, i.e. the speed half a tooth *before* the edge. MA and EMA with $\alpha = 2/(N+1)$ have the same lag, as predicted in 3.1 c).

b) Sketch (`code/solutions/wokwi/sheet03_rpmfilter/sketch.ino`), filter and gap handling:

```
void process(float r, bool gap)
{
    int16_t ri = (int16_t)(r + 0.5f);
    if (!filt_init) {                          /* pre-load with first value */
        for (uint8_t i = 0; i < N_MA; i++) ma_buf[i] = ri;
        ma_sum = (int32_t)ri * N_MA;
        ema = r;
        filt_init = true;
```

```
    }
    /* moving average: running sum, add newest, subtract oldest */
    ma_sum += ri - ma_buf[ma_idx];
    ma_buf[ma_idx] = ri;
    if (++ma_idx >= N_MA) ma_idx = 0;
    float ma = (float)ma_sum / N_MA;
    /* exponential smoother */
    ema += ALPHA * (r - ema);
```

```
        /* per-tooth speed: rpm = 1 / T[s] = 2e6 / T[ticks] */
        float r = 2.0e6f / T;
        uint8_t rep = 1;
#if GAP_MODE >= 1
        if (gap) r *= 3.0f;                    /* gap spans 18 deg */
#endif
#if GAP_MODE == 2
        if (gap) rep = 3;                      /* one sample per 6 deg */
#endif
        for (uint8_t i = 0; i < rep; i++) process(r, gap);
```

Expected capture (reference sketch run on the PC with simulated crank edges; step 3000→5000 rpm at $k = 0$):

```
k,raw,ma,ema,gap        k,raw,ma,ema,gap
-1,3012,3018,3014,0      7,4950,3994,4246,0
0,4988,3143,3247,0       8,4950,4114,4329,0
1,4988,3267,3451,0      14,4988,4846,4671,0
4,4963,3633,3926,0      15,5013,4971,4711,0
5,4950,3753,4046,0      18,5025,4978,4808,0
```

The MA rises linearly by $2000/16 = 125$ rpm per tooth and reaches the new value after 16 samples; the EMA jumps by $\alpha \cdot 2000 = 235$ rpm first and then approaches exponentially: 50 % after 5, 90 % after 18 samples (MA: 8 and 14). The ramp lag follows from the step response, because for an LTI system the response to a ramp is the running sum of the step response: $D = \sum_{k\geq 0}(1 - s[k])$ evaluated on the capture (baseline = mean of the 32 pre-trigger samples, final value = mean of the last 36) gives 7.65 (MA) and 7.56 (EMA) samples, close to 7.5; the extra half sample of the offline ramp (7.9) is hidden here because the time origin is the first *measured* jump. A staircase is a sum of shifted steps – by superposition its response is the sum of shifted step responses; nothing new is learned from it. Note: the per-revolution line samples the filters always at the same crank angle (at the gap), so the ripple appears there as a constant offset (e.g. MA_{16} 2982, EMA 2986 at 3000 rpm) – stroboscopic sampling of a crank-synchronous signal.

c) Mode 0: the gap value is $n/3$, i.e. a negative pulse of $-2n/3 = -2000$ rpm once per revolution. MA_{16} shows a rectangular dip of $2000/16 = 125$ rpm lasting 16 teeth (min 2860), the EMA a dip of $\alpha \cdot 2000 = 235$ rpm (min 2751) that decays over $\approx$ 20 teeth; the mean is biased by -1.1 % (compare 2.1 c). The per-revolution line, printed right after the gap, shows the worst value (3000 rpm: MA 2860, EMA 2751; 7000 rpm: 6674, 6418). Mode 1 removes the bias, but the sample covering 18° has the same weight as those covering 6°; MA_{30} then spans 30 samples $\neq 180°$ for half of the windows: residual std 1.06 rpm. Mode 2 makes the sample index proportional to the crank angle (6° per sample, 60 per revolution); MA_{30} spans exactly 180° and cancels the 2nd order completely (std 0.04). Time invariance: in the tooth domain the operator "tooth period → speed" must be the same for every index k. With the gap it is not: at $k \equiv 57 \pmod{58}$ the measurement covers 18° instead of 6°, i.e. the system changes periodically with k (like S1 or S10 of Sheet 2). A shift of the speed profile by one tooth does not shift the output by one sample. An LTI filter after a time-variant measurement produces artefacts that no filter design can remove; the correction must restore a uniform (angle-equidistant) sequence first.

d) At 1000 rpm one sample takes 1 ms; 5000 rpm/s are 5 rpm per sample. 0.5 % = 5 rpm allow a lag of only 1 sample, i.e. $(N-1)/2 + 0.5 \leq 1$, $N \leq 2$ – no ripple suppression at all ($|H| \approx 1$).

Conversely MA_{30} removes the ripple but lags 15.5 samples = 15.5 ms, i.e. 78 rpm (7.8 %). A causal smoother cannot have both. Real ECUs therefore (i) measure the *segment time* (time for $720°/z = 180°$, i.e. exactly the MA_{30} in the period domain) for torque/misfire functions, synchronous to the firing, (ii) use the short tooth time for angle extrapolation between teeth, and (iii) compensate the known lag by prediction ($\hat{n} = n_{\mathrm{filt}} + \dot{n} \cdot D$, an estimate of the acceleration). Note also that the lag in *samples* is constant while the lag in *time* scales with $1/n$: at 6000 rpm the same MA_{30} lags only 2.6 ms.

Deliverables

3.1: convolution table, derivations of h, s, noise gain, D, $|H|$. 3.2: program and output, explanation of the rounding and truncation errors, plot of the triangular impulse response. 3.3: completed `rpm_offline.c` with result tables; sketch; capture of a step for all three gap modes (plot in JSLinux), 50/90 % points and ramp lag; answer to d).